

# Practica - 12 Introducción a Python, R y Julia

Luis Gálvez

## Tabla de contenidos

|                                                                        |          |
|------------------------------------------------------------------------|----------|
| <b>1 Proposito de la practica</b>                                      | <b>1</b> |
| 1.1 Ejercicio 1: Hidrología del Río Magdalena (Vectores y Matemáticas) | 1        |
| 1.1.1 Desarrollo . . . . .                                             | 2        |
| 1.2 Ejercicio 2: Calidad del Aire en Bogotá (DataFrames y Filtrado)    | 3        |
| 1.2.1 Desarrollo . . . . .                                             | 3        |
| 1.3 Preguntas . . . . .                                                | 5        |
| 1.3.1 Desarrollo . . . . .                                             | 5        |

## 1 Proposito de la practica

### 1.1 Ejercicio 1: Hidrología del Río Magdalena (Vectores y Matemáticas)

**Contexto:** Estás analizando el comportamiento de las estaciones hidrológicas del IDEAM a lo largo del Río Magdalena. Has recibido el reporte del caudal medio mensual (en metros cúbicos por segundo,  $m^3/s$ ) de cinco estaciones clave y necesitas extraer estadísticos básicos y transformar las unidades para un informe ambiental.

#### Instrucciones de código:

1. Define una colección ordenada (Vector/Lista/Array) llamada `estaciones` con los siguientes nombres: "Honda", "Puerto Berrío", "Barrancabermeja", "Puerto Wilches", "Calamar".
2. Define otra colección paralela llamada `caudales_m3s` con los siguientes valores numéricos: 1500, 2100, 2800, 3200, 4500.
3. Utilizando las funciones estadísticas nativas de tu lenguaje elegido, calcula e imprime:
  - El **caudal máximo** registrado en el río.
  - El **caudal promedio** de las cinco estaciones.
4. Para un estudio local, necesitas el caudal en **litros por segundo (l/s)**. Utilizando el concepto de *vectorización* (o *broadcasting*), multiplica la colección `caudales_m3s` por 1000 y guarda el resultado en una nueva variable llamada `caudales_ls`. (Asegúrate de no usar bucles `for`).

5. Imprime la colección resultante `caudales_ls`.

### 1.1.1 Desarrollo

```
import numpy as np

#-----
# Colección de estaciones
estaciones = [
    "Honda",
    "Puerto Berrío",
    "Barrancabermeja",
    "Puerto Wilches",
    "Calamar"
]

#-----
# Colección de caudales en m3/s
caudales_m3s = np.array([1500, 2100, 2800, 3200, 4500])

#-----
# Estadísticos básicos
caudal_maximo = np.max(caudales_m3s)
caudal_promedio = np.mean(caudales_m3s)

print("Caudal máximo registrado:", caudal_maximo, "m3/s")
print("Caudal promedio:", caudal_promedio, "m3/s")

#-----
# Conversión a litros por segundo usando vectorización
caudales_ls = caudales_m3s * 1000

#-----
# Imprimir resultado
print("Caudales en litros por segundo:")
print(caudales_ls)
```

```
Caudal máximo registrado: 4500 m3/s
Caudal promedio: 2820.0 m3/s
Caudales en litros por segundo:
[1500000 2100000 2800000 3200000 4500000]
```

## 1.2 Ejercicio 2: Calidad del Aire en Bogotá (DataFrames y Filtrado)

**Contexto:** La Red de Monitoreo de Calidad del Aire de Bogotá ha publicado los datos promedio diarios de Material Particulado 2.5 ( $PM_{2.5}$ ). Debes organizar estos datos en una estructura tabular, filtrar las estaciones que presentan riesgos para la salud (según la OMS) y visualizar los resultados.

### Instrucciones de código:

1. Crea un **DataFrame** llamado `df_aire` (utilizando la librería adecuada según tu lenguaje: `pandas`, `data.frame` o `DataFrames.jl`) que contenga dos columnas:
  - `estacion`: "Carvajal", "Kennedy", "Fontibón", "Suba", "Usaquén"
  - `pm25`: 55, 42, 38, 15, 12
2. Imprime un **resumen descriptivo/técnico** de tu DataFrame (usando la función equivalente a `info()`, `str()` o `describe()`).
3. El límite diario recomendado por la OMS para  $PM_{2.5}$  es de  $15 \mu g/m^3$ . Crea un nuevo DataFrame llamado `df_alerta` que **filtre** y contenga únicamente las estaciones donde el nivel de `pm25` sea **estrictamente mayor a 15**.
4. Crea una **nueva columna** en `df_alerta` llamada `estado` y asígnale a todas sus filas el valor de texto "Crítico".
5. Utiliza la librería gráfica nativa de tu entorno para generar un **gráfico de barras** simple usando el DataFrame original (`df_aire`), donde el eje X sean las estaciones y el eje Y sean los niveles de  $PM_{2.5}$ .

### 1.2.1 Desarrollo

```
import pandas as pd
import matplotlib.pyplot as plt

#-----
# Crear DataFrame
df_aire = pd.DataFrame({
    "estacion": ["Carvajal", "Kennedy", "Fontibón", "Suba", "Usaquén"],
    "pm25": [55, 42, 38, 15, 12]
})

#-----
# Resumen descriptivo/técnico
print("Información del DataFrame:")
print(df_aire.info())

print("\nResumen estadístico:")
print(df_aire.describe())
```

```

#-----
# Filtrar estaciones con PM2.5 > 15
df_alerta = df_aire[df_aire["pm25"] > 15].copy()

#-----
# Crear nueva columna estado
df_alerta["estado"] = "Crítico"

print("\nEstaciones en alerta:")
print(df_alerta)

#-----
# Gráfico de barras
plt.bar(df_aire["estacion"], df_aire["pm25"])

plt.xlabel("Estaciones")
plt.ylabel("PM2.5")
plt.title("Niveles diarios de PM2.5 en Bogotá")

plt.show()

```

Información del DataFrame:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   estacion    5 non-null      object
1   pm25        5 non-null      int64
dtypes: int64(1), object(1)
memory usage: 212.0+ bytes
None

```

Resumen estadístico:

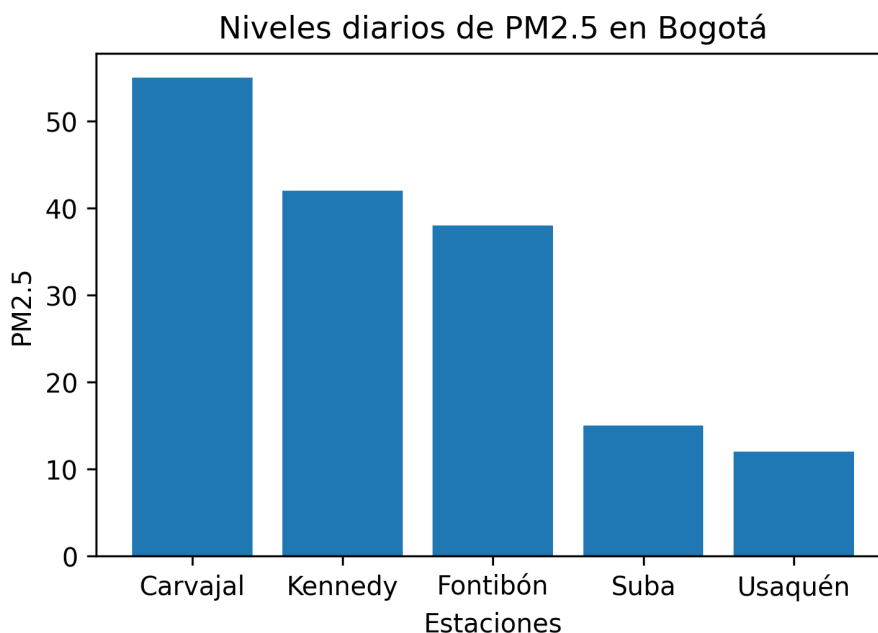
```

          pm25
count    5.000000
mean     32.400000
std      18.392933
min      12.000000
25%      15.000000
50%      38.000000
75%      42.000000
max      55.000000

```

Estaciones en alerta:

|   | estacion | pm25 | estado  |
|---|----------|------|---------|
| 0 | Carvajal | 55   | Crítico |
| 1 | Kennedy  | 42   | Crítico |
| 2 | Fontibón | 38   | Crítico |



### 1.3 Preguntas

1. **Sobre el Ejercicio 1 (Vectorización):** Intenta explicar qué sucedería si decides hacer este ejercicio en **Python** y creas `caudales_m3s` como una Lista nativa (`[1500, 2100...]`) y la multiplicas directamente por 1000 (`caudales_m3s * 1000`). ¿Realizaría la operación matemática deseada? ¿Qué librería de Python soluciona este problema y cómo se diferencia este comportamiento del de R o Julia?
2. **Pregunta General (Indexación):** Escribe la línea de código exacta que usarías en tu lenguaje elegido para extraer **las tres primeras estaciones** del DataFrame del Ejercicio 2 usando indexación por rangos (*slicing*). Explica brevemente si el límite superior del rango que escribiste se incluye o se excluye en el resultado final, justificando esto según el tipo de indexación (Base-0 vs Base-1) de tu lenguaje.

#### 1.3.1 Desarrollo

1. Si los caudales se definen como una lista nativa y se usa el operador `*` no se ejecutará una multiplicación de los elementos si no que se repetirá la

secuencia, resultando en una lista demasiado larga compuesta por valores repetidos. En python NumPy soluciona este problema al utilizar los array, sobre los cuales se ejecutan las operaciones vectorizadas, de manera que al usar `*` se obtiene el resultado esperado (Una multiplicación) en vez de repetir la lista. Un comportamiento distinto de R y de Julia, en donde los vectores y arreglos numéricos tienen un soporte matemático de manera nativa, por lo que no necesitan librerías adicionales para realizar la operación

2. En Python, usando pandas se usa `“df_aire.iloc[0:3]”` para extraer las tres primeras estaciones. Este rango devuelve las filas con índices 0, 1 y 2, correspondientes a las tres primeras estaciones. Esto debido a que python cuenta el 0 como el primer índice y no cuenta el techo de la función slice, es decir que ignora el 3 y toma los índices 0, 1 y 2